

From sophisticated substrates to legible products

A concrete external-legibility package: repository patches, a rebuilt demo doctrine, an executable Beads launch cut, a single-machine swarm plan, visual prototypes, and sixteen parallel fork missions.

01 A story a stranger remembers

02 Proofs with independent oracles

03 A launch cut agents can execute

04 Sinex-backed Polylogue direction

Prepared from static repository access · 10 July 2026

The products are deeper than the story available to outsiders.

Capability is not the limiting factor. Category, narrative order, proof design, and first-run experience are.

Polylogue today

A sophisticated evidence model is often encountered as a broad transcript archive. The generated site drifts toward “AI memory,” and the current tour proves construct breadth before showing a memorable user problem.

Sinex today

Its most distinctive ideas—material versus interpretation, three clocks, replay, explicit gaps, and authority—sit behind service topology. The deterministic demo is a DB/API smoke path, not a thesis proof.

Joint risk

A metadata-only bridge wastes the fit; a generic-event merger destroys Polylogue’s domain value. The integration needs a strong authority matrix and stable identity/revision contract.

External legibility is a product surface: category → story → evidence → caveat → drill-down → next action.

One stable category sentence per project.

Everything else should reinforce these statements, not compete with them.

POLYLOGUE

**The local flight recorder
and system of record
for AI work.**

Claims have receipts. History has topology. Memory requires judgment.

SINEX

**The local evidence substrate
for digital life
and agent work.**

Preserve source material. Reinterpret honestly. Expose what is missing.

Together: Polylogue explains AI work; Sinex preserves the wider evidentiary world in which it happened.

Lead with one evidence chain, not an inventory.

The public-safe landing prototype makes the structural tool result the hero.

Polylogue

Why

Demos

Claims

Architecture

GitHub ↗

LOCAL · CROSS-PROVIDER · EVIDENCE-ADDRESSABLE

The flight recorder for AI work.

Search every agent history. Read tool activity as work, not chat. Audit claims against outcomes. Resume from reviewed evidence.

Run the private-data-free tour

See the evidence model

```
$ nix run github:Sinity/polylogue -- demo tour
```

evidence receipt · codex-session

Repair the failing parser test

codex-session

Run the test and stop if it fails.

\$ pytest tests/missing_test.py

FAILED · exit 4

ERROR: file or directory not found:
tests/missing_test.py

I hit an error and need the missing path corrected before continuing.

structural result

 →

message ref

raw block

lineage

01 / EVIDENCE

Claims have receipts
Resolve "tests pass" to the command, typed result, exit status, duration, and original provider block.

02 / TOPOLOGY

History has branches
Compose forks, continuations, subagents, and compactions without deleting physical evidence or double-counting replay.

03 / MEMORY

Memory requires judgment
Keep candidate claims separate from accepted lessons, then record exactly what context reached the next agent.

Deterministic tour: 11 sessions · 43 messages · 30/30 declared constructs verified

no private data · no API key · full receipts

Receipts, not rhetoric

Failure is grounded in exit_code=4 and a typed tool result.

Topology, not folders

Forked history composes without erasing physical evidence.

Judged memory

Candidates and accepted assertions remain different states.

Lead with epistemic payoff before infrastructure.

The strongest story is reconstruction plus an honest missing-source boundary.

SinexEVIDENCE SUBSTRATE

LOCAL · REPLAYABLE · SOURCE-AWARE

Know what happened—and what you do not know.

Sinex preserves source material, records versioned interpretations, exposes capture gaps, and joins digital activity without silently turning inference into fact.

A parser can change its mind. A source can go missing. A model can propose. None of those should erase the evidence or acquire authority by accident.

Explore the evidence model

Run the walkthrough

moment - 14:31:40–14:34:10coverage: partial

What was happening around the failed build?

14:31:52

cargo test returned exit 101

terminal:event/01 · material:atuin/88

14:32:08

Research: tokio test clock behavior

browser:event/12 · material:history/33

14:33:26

Modified tests/runtime_clock.rs

git:event/09 · repository:sinex

14:32–34

Evidence gap: focus source sequence 3812–3818 was not captured. No claim is made about the active window in this interval.

terminal

browser

git

coverage gap

01

Material ≠ interpretation

Keep the acquired bytes separate from what a parser inferred, so reinterpretation remains auditable.

02

Three clocks

Occurrence, coining, and persistence times remain distinct across late import and replay.

03

Confidence ≠ authority

Models propose. Judgment or declared policy promotes. Both remain inspectable.

04

Silence ≠ absence

Disabled, stale, partial, and empty sources produce different evidence states.

Rust · PostgreSQL · NATS · NixOS

source material → interpretation → projection → product

Material ≠ interpretation

Parser mistakes remain reversible.

Three clocks

Occurrence, coining, and persistence do not collapse.

Confidence ≠ authority

Models propose; judgment promotes.

Silence ≠ absence

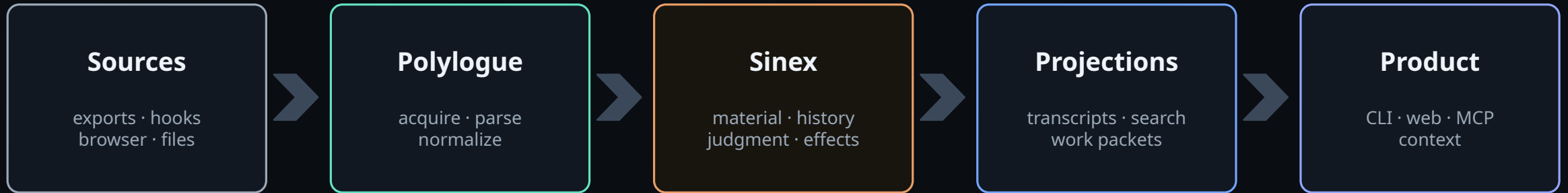
Disabled, stale, partial, and empty are distinct.

POLYLOGUE × SINEX · EXTERNAL LEGIBILITY KIT

5

Sinex is the durable backend; Polylogue remains the domain kernel.

The target stores transcripts in Sinex without flattening Polylogue into generic events.



Sinex owns durability

Provider-native + normalized material, replay-specific interpretations, judgments, lifecycle, coverage, settlement, and model effects.

Polylogue owns meaning

Sessions, messages, tools, lineage, compaction, physical/logical accounting, memory policy, query behavior, and UX.

SQLite remains valuable

Standalone mode, local/offline projection, FTS/vector acceleration, UI state, cache, watermarks, and outbox.

Decisive proof: drop rebuildable SQLite tiers and reconstruct them from Sinex-held material and history.

A demo is a bounded experiment, not a pretty transcript.

Every promoted story needs an oracle, negative controls, caveats, and a product-boundary receipt.

01 QUESTION

What decision or doubt does the viewer have?

02 CLAIM

What exactly will the artifact establish?

03 CONSTRUCT

Which property is being measured?

04 ORACLE

What independently determines the expected result?

05 FALSIFIERS

What outcome would make the claim fail?

06 RECEIPTS

Which refs, commands, packets, and hashes reproduce it?

Presentation order: payoff → evidence → honest limitation → drill-down → architecture reveal.

A small public arc, backed by deeper proof cards and torture tests.

The hero stories stop competing with subsystem verification.

HERO	Polylogue · The Receipts	assistant claim → typed failure → repair → safe resume
HERO	Sinex · Reconstruct Tuesday	multi-source moment + deliberate coverage hole
HERO	Joint · Resume This Bead	intent + agent work + machine effects + reviewed context
PROOF	History Has Branches	physical artifacts ≠ logical unique work
PROOF	The System Changes Its Mind	parser-v1 → replay → parser-v2 with history intact
TORTURE	Crash, resume, no duplicate occurrence	interrupted import with settlement and oracle

Keep the existing broad demo catalogs—but classify them as proof cards, experiments, or operational campaigns.

A deliberately narrow sequence drawn from existing Beads.

Trust and web reliability first; one flagship story and one anti-grep proof; then packaging and media.

0	TRUST FLOOR	0hqs · bby.1	Bound queries; truthful timeout/degraded UI
1	LANDING	3tl.12 · 3tl.15 · 3tl.16 · 3tl.8	README, anti-grep, claims ledger, repository surface
2	VISIBLE PRODUCT	ap7 · 212.2	Semantic tool cards + The Receipts flagship
3	EVIDENCE SHELF	3tl.4 · reuse 212.8	Bounded findings + honesty anti-demo
4	RELEASE	3tl.7 · 3tl.9 · 3tl.10	Clean install matrix, media drift gate, launch packet

Do not block the launch on the full Sinex backend, all semantic card types, or publishable memory uplift.

Parallelize by file ownership and resource class—not by issue count.

Sacrifice Git elegance for throughput; retain semantic authority and validation gates.

CHEAP / PARALLEL**Narrative · docs · claims · ADR**

6–10 agents

MEDIUM**fixtures · tour · renderer contracts**

3–5 agents

HEAVY TOKEN 1**full Polylogue verify / scale run**

one at a time

HEAVY TOKEN 2**Rust build / Sinex infra demo**

one at a time

BROWSER TOKEN**E2E / screenshots / media**

one at a time

CONDUCTOR LOOP

- 1 Assign exclusive paths
- 2 Require a proof packet
- 3 Cherry-pick or apply patch
- 4 Run narrow gate
- 5 Integrate by wave
- 6 Run cold-reader + privacy audit

Use worktrees for isolation; use a shared artifact shelf and merge packet for coordination.

The package can be executed as a conversation swarm.

Each prompt owns a bounded surface and returns a patch plus evidence.

PUBLIC STORY	01 landing · 05 claims · 06 install/media
POLYLOGUE PRODUCT	02 semantic cards · 03 Receipts · 04 anti-grep · 07 web · 08 resume · 09 lineage
SINEX PRODUCT	10 narrative · 11 moment · 12 replay · 13 outage
JOINT ARCHITECTURE	14 backend ADR · 15 Agent Work Packet
INTEGRATION	16 conductor / cold-reader / launch audit

Prompts are self-contained: scope, design constraints, negative controls, validation, and deliverables.

The package includes bounded repository patches, not just recommendations.

They are static-access improvements designed to be applied or mined into the active branches.

Polylogue patch

- category-aligned README and package metadata
- new Why / Demos / Public Claims / Sinex backend docs
- revised docs navigation and landing page
- evidence-first deterministic tour
- focused unit tests and generated-site checks

Sinex patch

- payoff-first README
- Concepts / For Agents / Demos / Public Claims docs
- transcript-complete Polylogue backend direction
- current bridge explicitly labeled transitional
- metadata-only doctrine corrected at the design layer

Visual prototypes

Polylogue landing · Sinex landing · Resume This Bead dashboard

Machine-readable plans

claims ledger · demo portfolio · Beads launch cut · worktree lanes

Proof artifact

A passing, private-data-free Polylogue tour with 30/30 declared constructs.

The shortest credible path

One memorable incident.
One anti-grep proof.
One honest limitation.
One reviewed resume.

Then let the architecture reveal why the result is trustworthy.

Artifacts: plans · patches · prompts · mockups · proof packet

SHIP BAR

- | | | |
|---|-------------------------------|------------------|
| ✓ | Bounded web failure states | must pass |
| ✓ | Flagship deterministic packet | must pass |
| ✓ | Claims resolve to receipts | must pass |
| ✓ | Clean install path | must pass |
| ✓ | Cold-reader category recall | must pass |
| – | Full Sinex backend | may follow |
| – | General memory uplift | experiment later |